THE NATIONAL UNIVERSITY OF LESOTHO

Department of Mathematics and Computer Science

CS3520 - Computer Organisation and Architecture I

Lab 1

3rd September 2025

1. Aim

To introduce students to basic RISC-V assembly programming concepts through a hands-on exercise, including a "Hello World" program and simple arithmetic operations, using the Ripes simulator.

By the end of this lab, students should be able to:

- Install and run the Ripes simulator.
- Understand the structure of a simple RISC-V assembly program.
- Write, assemble, and execute a program that outputs text.
- Implement arithmetic operations (addition, subtraction, multiplication, division) in RISC-V assembly.
- Develop confidence in editing and testing simple assembly programs.

2. System Requirements

- A personal computer (Windows, macOS, or Linux).
- Minimum 4GB RAM, 2GHz processor recommended.
- Operating system: Windows 10+, macOS 10.15+, or any modern Linux distribution.
- Internet access to download and install the Ripes simulator.

3. Theoretical Background

Assembly programming provides direct interaction with a computer's instruction set architecture (ISA). RISC-V is a modern open-source ISA designed for teaching and research.

- Registers are named x0 to x31 (with aliases like a0, t0, sp, etc.).
- Instructions follow a load-store architecture (only load/store interact with memory).
- Arithmetic instructions operate directly on registers, e.g.

```
add t0, t1, t2 # t0 = t1 + t2
```

System Calls

In most environments (including Ripes), programs interact with the operating system through system calls, triggered by the instruction ecall.

- The system call number is placed in register a7.
- Arguments are placed in registers a0, a1, etc.
- The kernel executes the requested service and, if applicable, places the result back in a0.

This mechanism allows assembly programs to do tasks like printing, reading input, or exiting the program.

4. Lab Exercises

4.1 Exercise 1 - Hello World

Write and run a RISC-V assembly program to display Hello World in the console.

```
.data msg:
.asciz "Hello World\n" # Null-terminated string
.text
.globl _start
_start:
    la a0, msg # Load address of string into a0
    li a7, 4 # System call: print string ecall # Make the system call
    li a7, 10 # System call: exit
    ecall
```

4.2 Exercise 2 - Factorial Calculator

```
.data
    prompt: .asciz "Enter a positive integer: "
    error: .asciz "Error: Please enter a non-negative integer.\n"
    result: .asciz "The factorial is: "
.text
.globl _start _start:
    # Print prompt
    la a0, prompt
    li a7, 4
    ecall
    # Read integer input
    li a7, 5
    ecall
    mv t0, a0 # store input n in t0
    # If n < 0, print error and exit
    blt t0, x0, print_error
    # Initialize factorial = 1
    li t1, 1 # factorial result
    li t2, 1 # counter i = 1
    loop:
        bgt t2, t0, done # if i > n, exit loop
        mul t1, t1, t2 # factorial *= i
        addi t2, t2, 1 # i++ j loop
done:
    # Print result label
    la a0, result
    li a7, 4 ecall
# Print factorial result
    mv a0, t1
    li a7, 1
    ecall
    j exit
print_error:
    la a0, error
    li a7, 4
    ecall
exit:
    li a7, 10
    ecall
```

4.3 Exercise 3

Write RISC-V assembly programs that:

1. **Grain Counting (Makopokopo):** Given a quantity of grains in litres, determine and display how many makokopo (20L) it makes and how much remains.
2. **Cattle Wealth (Mahali):** Input number of cattle and classify the owner as "Not enough for bohali", "Small herd", or "Wealthy cattle owner".
3. **Basotho Hat Pattern (Mokorotlo):** Input height n and print a triangle pattern with * repeated up to n lines.
4. **Sesotho Calendar Days:** Input a number 1–7 and output the corresponding Sesotho day name, else print an error.